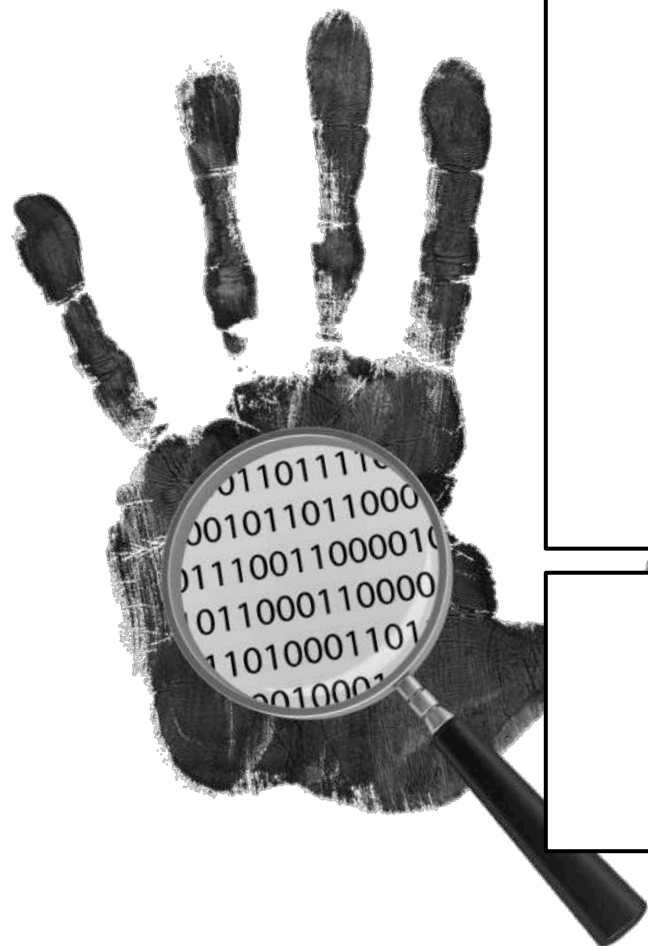




Storage and Memory Analysis

Part II. Techniques and Tools for Computer Analysis



CSF: Forensics Cyber-Security

Fall 2017

Nuno Santos



Previous classes



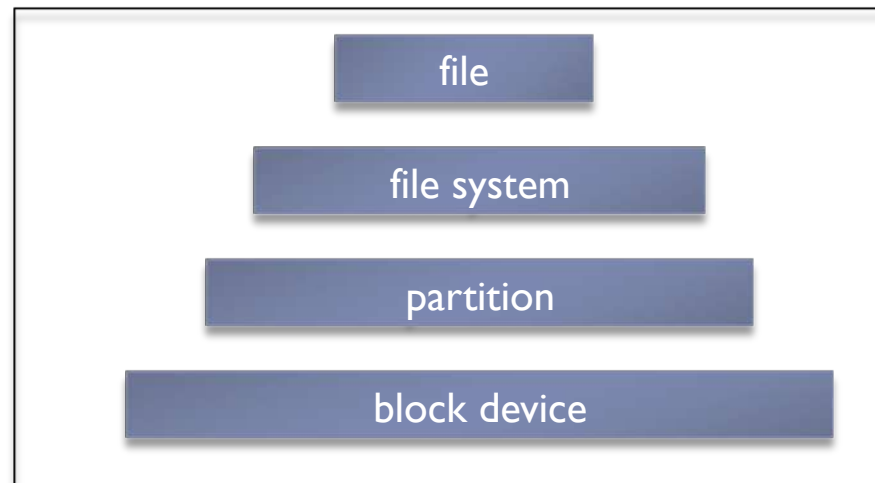
Files, steganography, watermarking





The storage abstraction layer

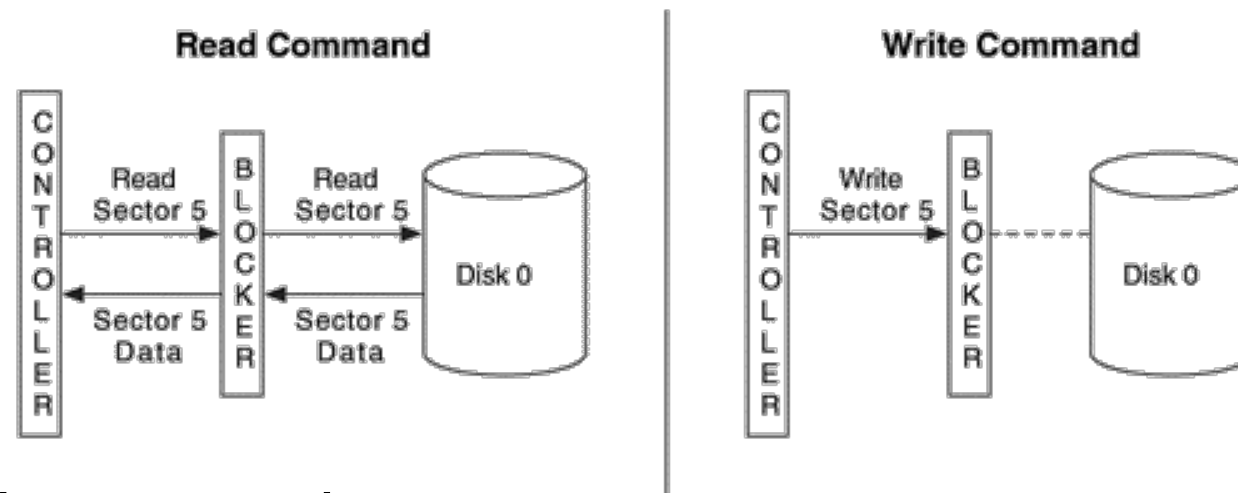
- ▶ Data can be organized in different **abstraction layers**



Abstraction layers of a typical storage stack

- ▶ When performing forensics investigation:
 - ▶ We can focus on abstraction layers independently
 - ▶ Get information of higher layers, by looking at lower layers

- ▶ Copying to keep the original untouched
 - ▶ Write blockers are frequently used to prevent changes



- ▶ Read the source data
 - ▶ Raw copy, errors on sector level copied as zeros
- ▶ Writing the data
 - ▶ To an image file or to another medium



Class roadmap

- ▶ Storage media
- ▶ Volume analysis
- ▶ Memory analysis

Storage media



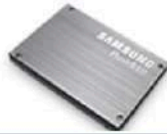
Forensically relevant issues on storage media

- ▶ Implications of storage technology for data acquisition
- ▶ Potential locations for data hiding
- ▶ Hardware restrictions to data access
- ▶ The particular case of mobile devices



COMMON STORAGE TECHNOLOGY

Solid State Drives (SDD) and Hard Disk Drives (HDD)

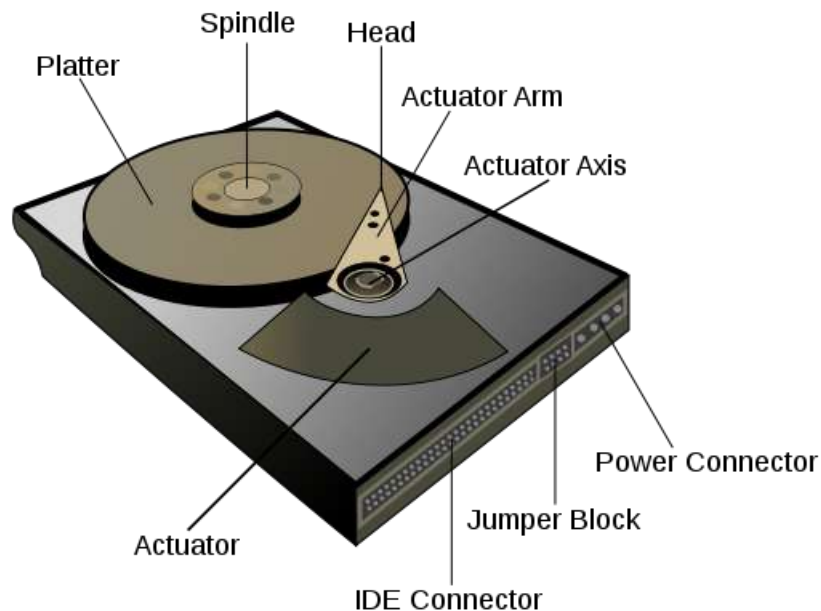


2.5" SATA 3.8Gbps SSD		2.5" SATA 3.8Gbps HDD
Solid NAND Flash based	Mechanism type	Magnetic rotating platters
64GB	Density	80GB
73g	Weight	365g
Read: 100MB/s Write: 80 MB/s	Performance	Read: 59MB/s Write: 60MB/s
1W	Active Power Consumption	3.86W
20G (10~2000Hz)	Operating Vibration	0.5G (22~350Hz)
1500G for 0.5ms	Shock Resistance	170G for 0.5ms
0 °C–70°C	Operating temperature	5°C– 55°C
None	Acoustic Noise	0.3 dB
MTBF > 2M hours	Endurance	MTBF < 0.7M hours



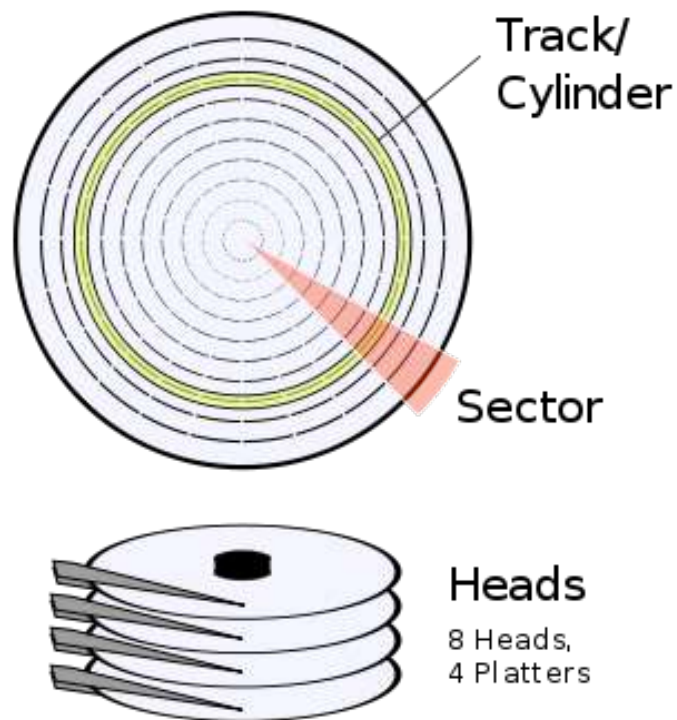
Hard disks

- ▶ Most forensic data is stored on hard disc drives
 - ▶ In commercial use since 1956





Hard disk basic terminology



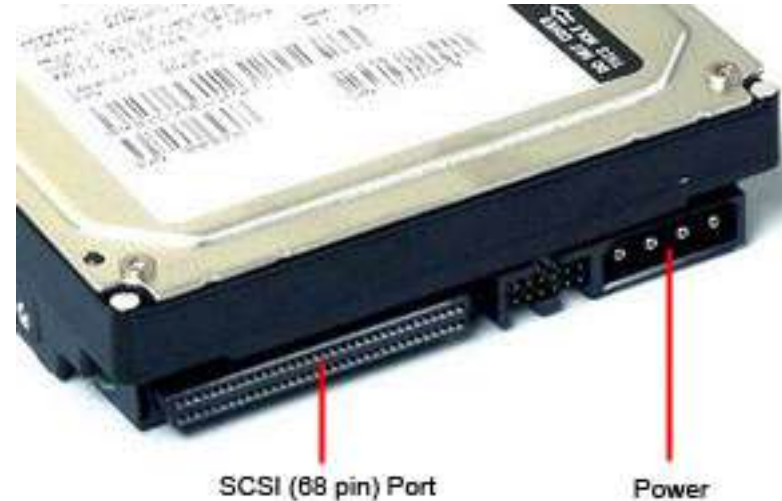
- ▶ **Head**
 - ▶ Device which reads and writes data on the disk
- ▶ **Track**
 - ▶ Individual circles on disk platter where data are located
- ▶ **Cylinder**
 - ▶ A column of tracks on a disk drive with 2 or more platters
- ▶ **Sector**
 - ▶ An individual section of data on a track – the smallest amount of data which can be written to the disk – usually 512 bytes
- ▶ **Disk Capacity = #cylinders * #heads * #sectors * sector_size**



ATA / IDE and SCSI hard disk interfaces



ATA interface



SCSI interface

- ▶ The ATA interface is (was) the most popular hard disk interface
- ▶ SCSI is an interface standard usually found in servers



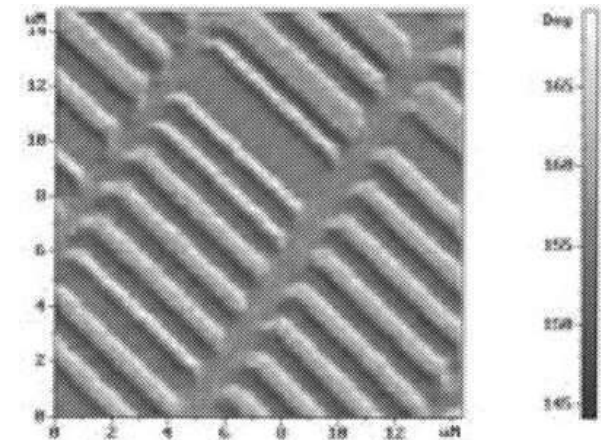
Disk addressing schemes

- ▶ **Cylinder, Head, Sector (CHS)**
 - ▶ Closely tied to the physical geometry of the disk drive
- ▶ **Logical Block Address (LBA)**
 - ▶ Independent from the physical geometry of the disk drive
 - ▶ First block on disk numbered 0, next is 1, ...
 - ▶ Most modern drives use this scheme



Hard disk formatting

- ▶ **Low-level formatting**
 - ▶ Physically defines tracks and sectors on disk
 - ▶ Does erase data
 - ▶ Typically only performed at factory
- ▶ **High-level formatting**
 - ▶ Performed when initializing a file system on a partition
 - ▶ Does not destroy data on disk





For forensics, hard drives are well understood

- ▶ Bits of data are placed onto magnetic media via repositionable recording heads
- ▶ The data may be randomly accessed by moving the heads over a selected cylinder
- ▶ All such operations are easily controlled via drive control commands which, for example, allow a sector of information to be read or written
- ▶ All of these are nice properties for forensic analysis, but for SSDs things get tougher...



Solid State Disk technology

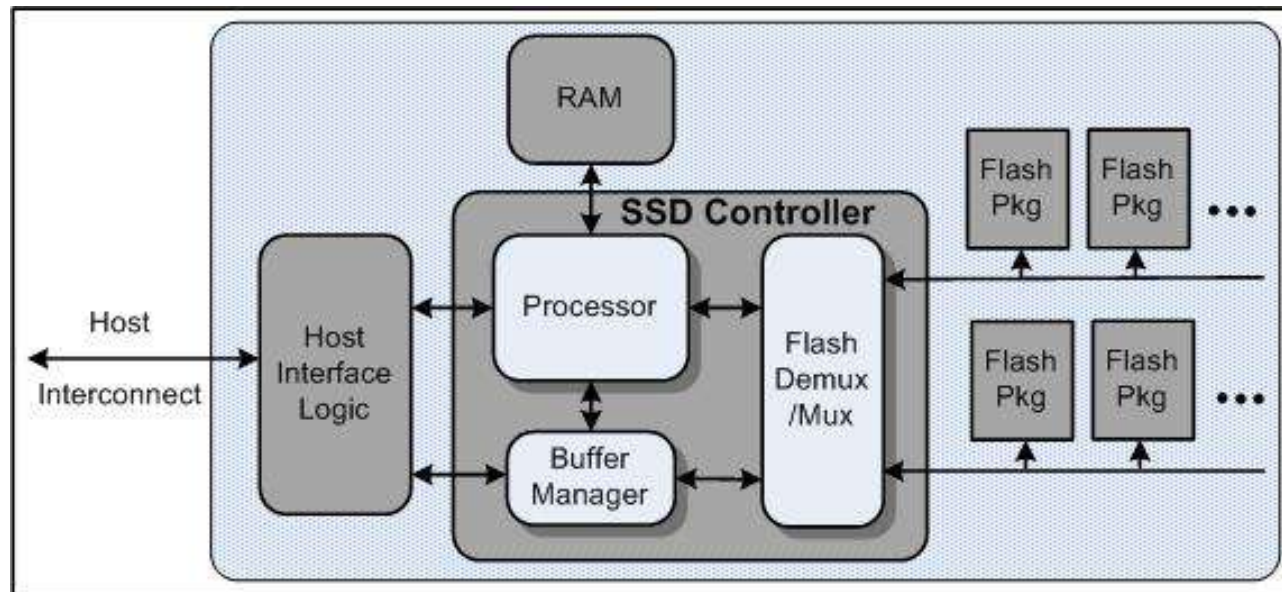
- ▶ Solid-state disks (SSDs)
- ▶ Rather than spinning platters, these drives use flash memory chips to store data





Internal structure of an SSD

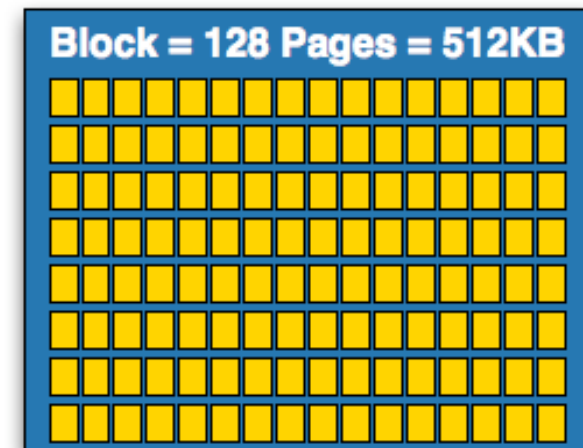
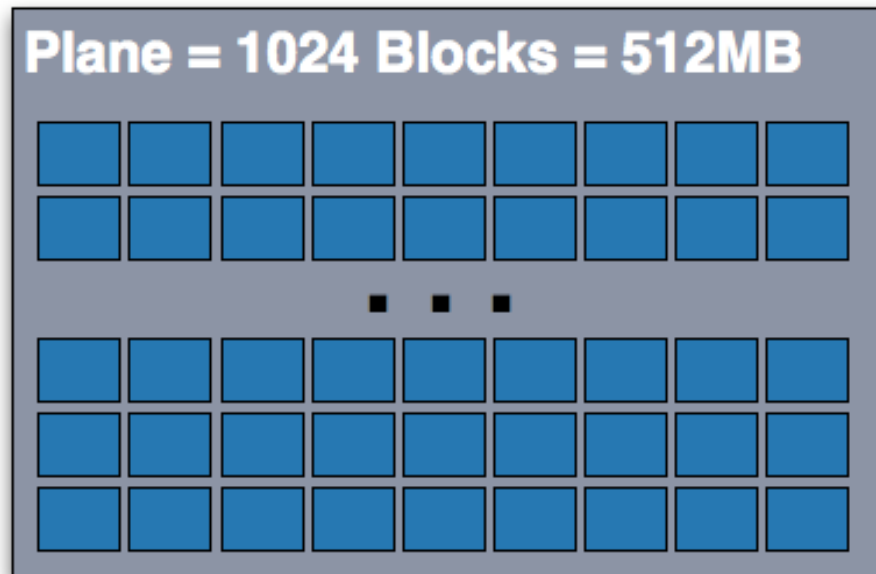
- Data is permanently stored in flash memory





Internal structure of the flash memory

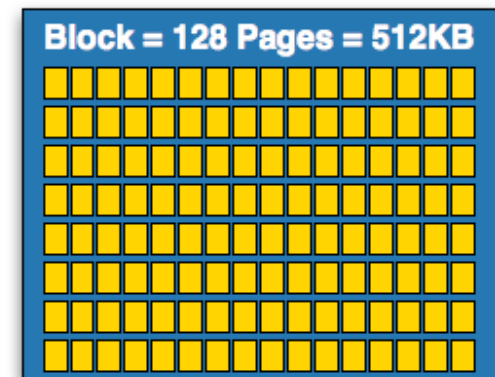
- Components: planes, blocks, and pages





Overview of SSD operations

- ▶ SSDs get slower as they fill up
- ▶ The smallest structure you can **write** is a **page** (4 KB)
- ▶ But you cannot write to a page unless it is empty
- ▶ The smallest structure you can **erase** is a **block** (512 KB)
- ▶ Also, you can only erase a block 10,000 times before it fails





Wear leveling

- ▶ If a particular block was programmed and erased repeatedly without writing to any other blocks, that block would wear out before all the other blocks
- ▶ For this reason, SSD controllers use a technique called **wear leveling** to distribute writes as evenly as possible across all the flash blocks in the SSD
- ▶ SSDs are not all the same—they make different attempts to increase lifespan with wear leveling



Garbage collection

- ▶ SSD write performance is significantly impacted by the availability of free, programmable blocks
- ▶ TRIM command enables the OS to tell the SSD controller that a file has been deleted
- ▶ The SSD controller can then implement a garbage collection algorithm to efficiently reclaim blocks



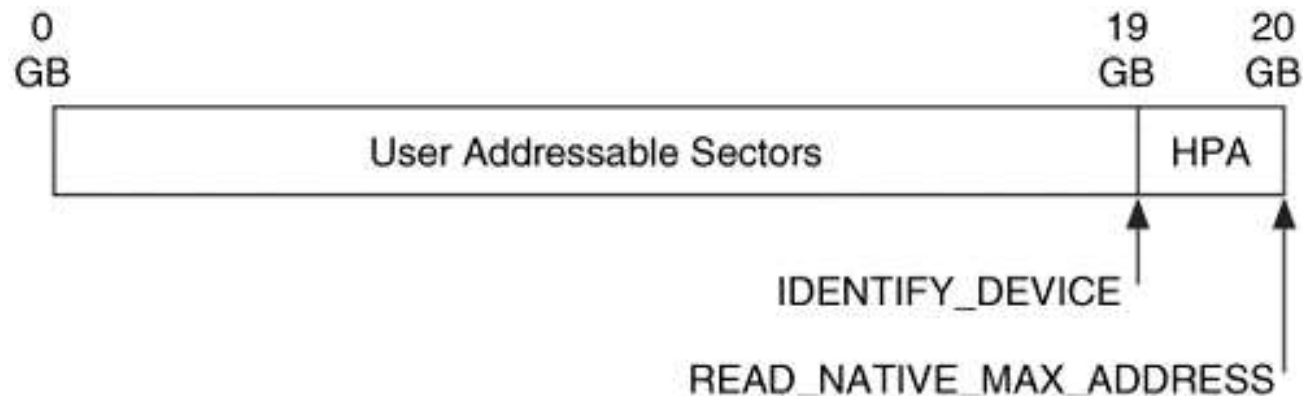
Challenges of SSDs for forensic investigators

- ▶ SSD's internal data structures are hidden
 - ▶ Internals not well understood - no accepted standards
- ▶ The physical location of any block within the SSD device will almost certainly not match the external Logical Block Address
- ▶ Wear leveling algorithms fragment data on the drive, but in an unpredictable way (non-standard)
 - ▶ May distribute multiple copies of data in various locations
- ▶ Garbage collection clears blocks marked for deletion
 - ▶ When an SSD is powered on, it may automatically initiate a trim operation to clear deleted data



Potential locations for data hiding in HDDs

- ▶ Host Protected Area (HPA) was added in ATA-4 spec
 - ▶ Computer vendors can store data that would not be erased when a user formats the HDD
- ▶ Can be detected by comparing output of ATA commands
 - ▶ An HPA can contain system files, hidden information, or both





Hard disk passwords

- ▶ ATA-3 spec introduced optional security features
- ▶ Passwords can be set to lock the HDD against reading / writing
- ▶ Data recovery is still possible by opening the disk
- ▶ Password can be used to wipe the disk



Self-encrypting drives

- ▶ Hard drive firmware includes encryption
- ▶ Custom firmware simulates unencrypted boot partition
 - ▶ But the entire user-accessible portion of the disk is encrypted
- ▶ User must enter a key to boot up
- ▶ Forensic erase takes less than a second
 - ▶ Simply overwrite the key



Self-wiping hard drives

- ▶ Wipes out key when drive is moved to another computer
- ▶ Makes traditional acquisition impossible

MKxx61GSYG Series Hard Disk Drive

160 / 250 / 320 / 500 / 640GB*
2.5-Inch / 7,200 RPM / SATA

TOSHIBA
Leading Innovation >>>



Secure mobile client-class disk storage designed for client PC and multifunction printer systems where host authentication is a critical aspect of maintaining the security of stored user data.

- AES-256 Bit Hardware-based Self-Encrypting Drive
- Toshiba Wipe Technology
- Trusted Computing Group Storage Security Subsystem Class: Opal SSC
- Secure Automatic Data Invalidation (ADI)
- Supported by Third Party Security Software Applications
- Up to 640GB* of Storage Capacity
- 9.5-millimeter High Profile
- 16MB Cache Buffer



Data extraction from mobile devices

- ▶ Tend to be very hard without knowing access credentials
- ▶ If device is rooted, that's the best
 - ▶ Custom ROMs install a modified recovery partition which offer shell access with root privileges from the recovery console
 - ▶ User data partitions can be mounted read-only and use imaging tool
- ▶ Backups
- ▶ There are hardware-based physical techniques



Mobile HW-based physical techniques: JTAG

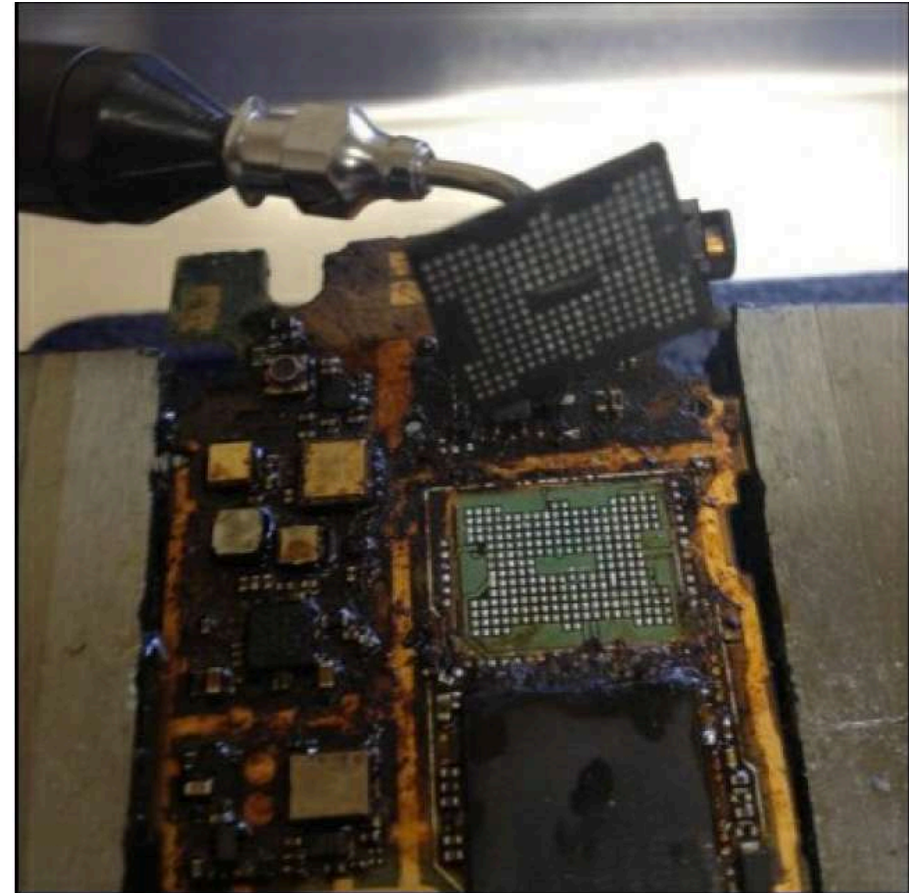
- ▶ JTAG interface: used during the device production process to communicate with the processor for testing
 - ▶ Allows examiners to communicate directly with the processor and retrieve a full physical image of the flash memory





Mobile HW-based physical techniques: Chip-off

- ▶ Chip-off involves heating the device's circuit board until the solder holding the components to the board melts, and then removing the flash memory chip
- ▶ The memory chip can then be read using commercial tools, resulting in a full physical image



Volume analysis



Volumes and partitions

- ▶ A **volume** is a collection of addressable sectors that an OS or application can use for data storage
 - ▶ Sectors need not be consecutive on a physical storage device
 - ▶ A volume may result from assembling / merging smaller volumes

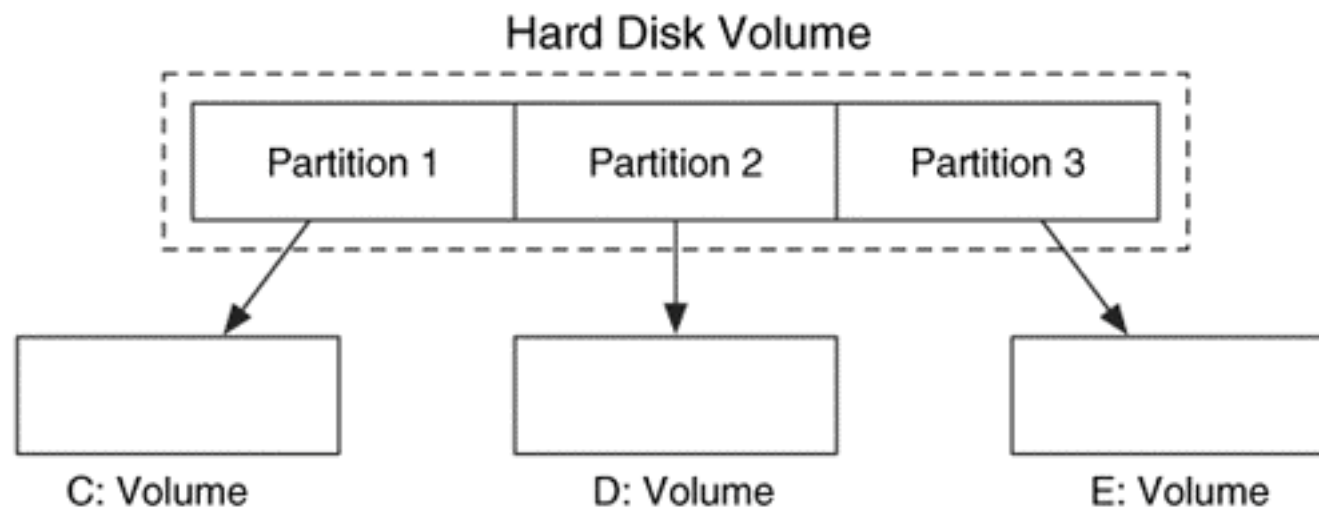
- ▶ A **partition** is a fraction of consecutive sectors in a volume. By definition, a partition is also a volume

- ▶ Partitions are used in many scenarios, including
 - ▶ Some file systems have max size smaller than hard disks
 - ▶ Many laptops put to sleep store memory on special partition
 - ▶ Separate partitions for booting multiple OSes



Volumes and partitions

- ▶ Consider a Microsoft Windows system with one hard disk
 - ▶ The hard disk volume is partitioned into three smaller volumes
 - ▶ Each volume has a file system
 - ▶ Windows assigns the names C, D, and E to each volume





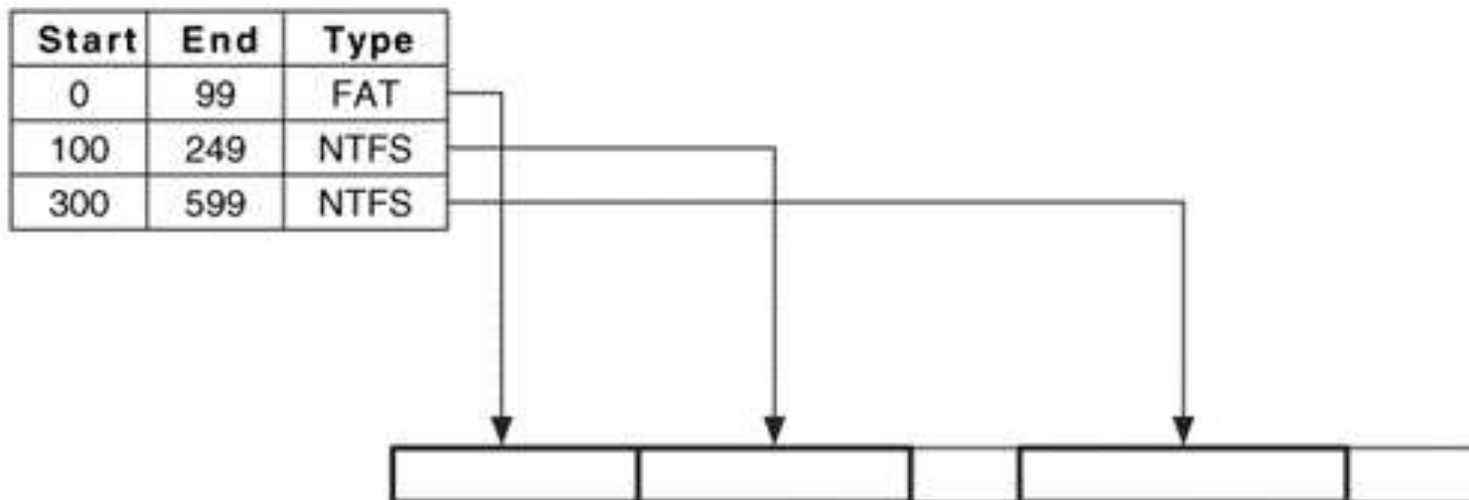
Basic analysis of volume layout

1. In many cases, an investigator **acquires** an entire hard disk and imports the image into his analysis software
2. To identify the volume **layout**, where the file system starts and ends, the partition tables must be analyzed
 - ▶ Not all sectors need to be assigned to a partition, and may contain data from a previous FS or that the suspect was trying to hide
3. In some cases, the partition system may become **corrupt** or **erased**, and automated tools will not work



Be aware of partitioning methods

- ▶ OS and hardware platform use different partitioning methods
- ▶ Typical partition systems have tables; entries describe partitions
 - ▶ Table entry has the starting sector, ending sector, and the type of partition

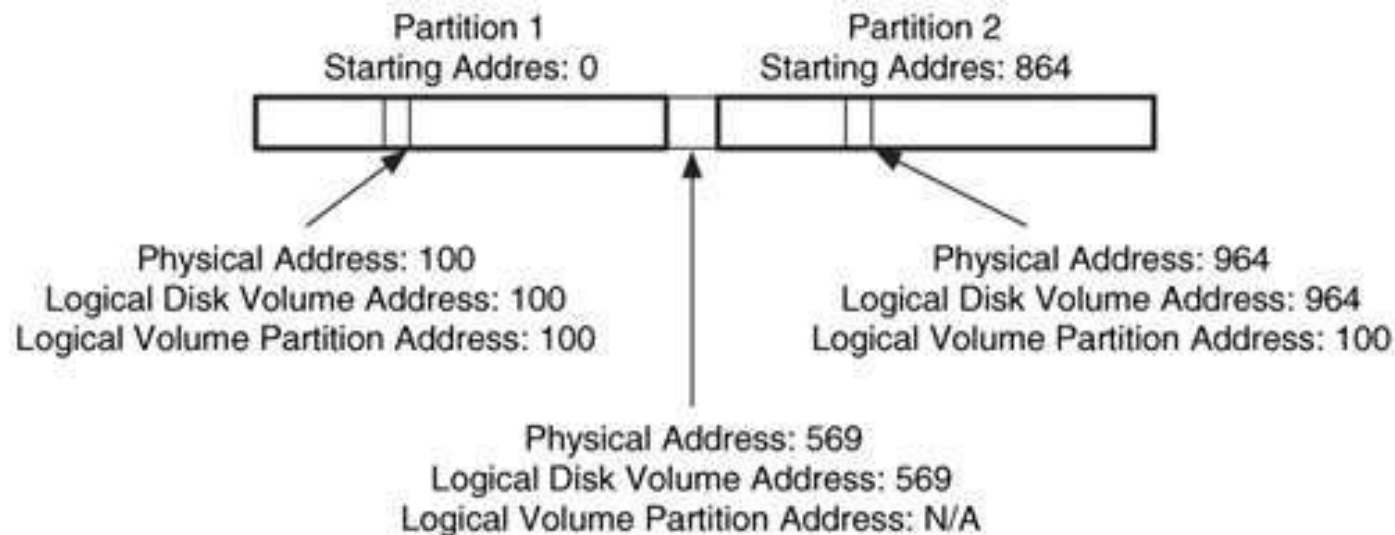


- ▶ A partition system cannot serve its purpose if those values are corrupt or non-existent



Sector addressing

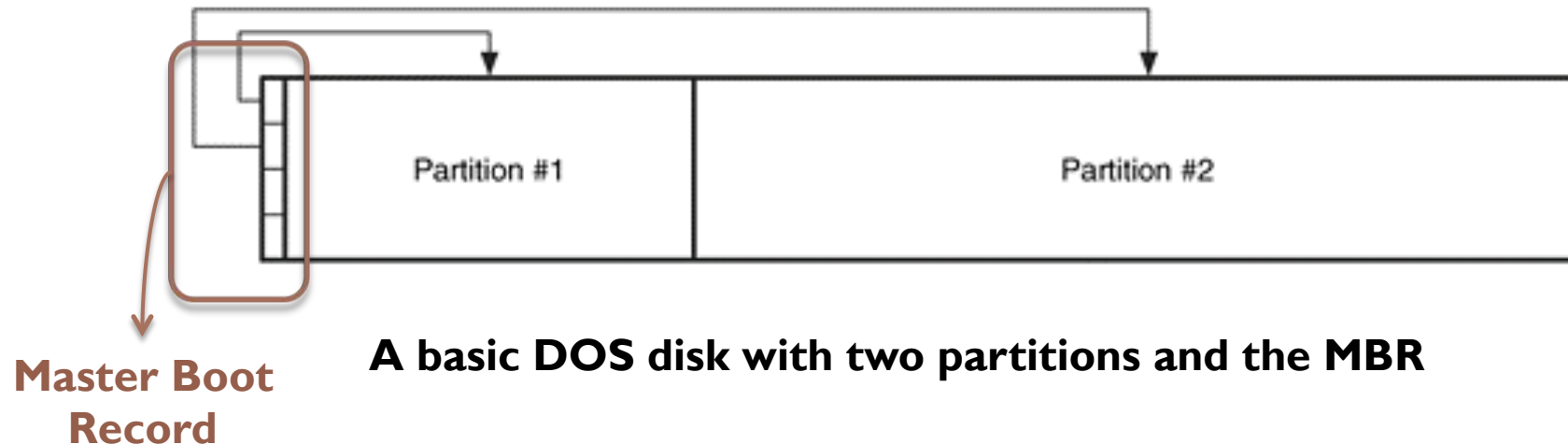
- ▶ Common method to address a sector: LBA address
 - ▶ LBA = Logical Block Addresses
 - ▶ A number that starts at 0 at the first sector of the disk
- ▶ A sector may have several addressing systems





A prevalent partition system: DOS partitions

- ▶ A disk that is organized using DOS partitions has a Master Boot Record (MBR) in the first 512-byte sector
- ▶ The MBR has a partition table with four entries, one for each partition



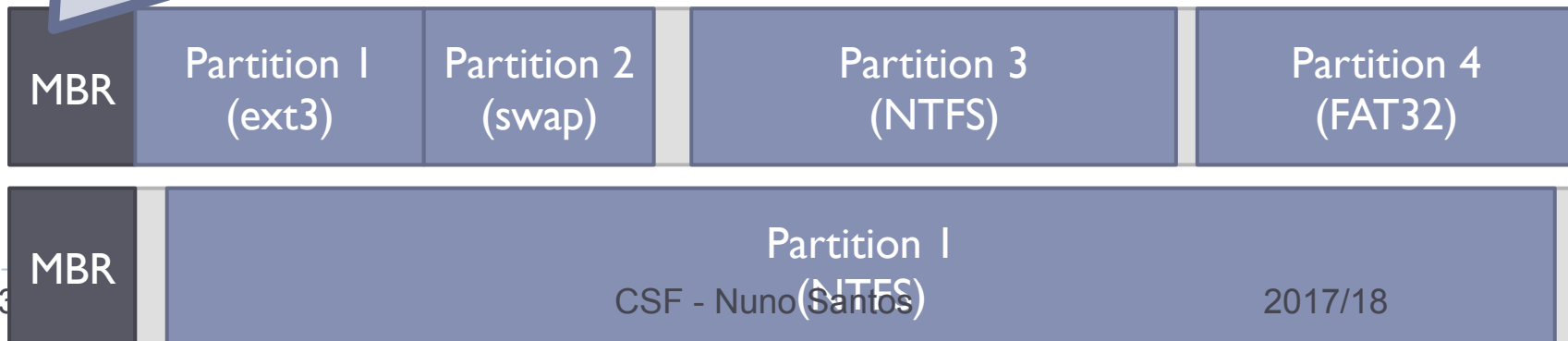


MBR: High level example

Address		Description	Size (Bytes)
Hex	Dec.		
0x000	0	Bootstrap code area	446
0x1BE	446	Partition Entry #1	16
0x1CE	462	Partition Entry #2	16
0x1DE	478	Partition Entry #3	16
0x1EE	494	Partition Entry #4	16
0x1FE	510	Magic Number	2
Total:			512

Includes the starting LBA and length of the partition

Disk 1
Disk 2





MBR layout

- The MBR contains boot code, a partition table, and a signature value

Structure of a classical generic MBR

Address		Description	Size in bytes
Hex	Dec		
+000h	+0	Bootstrap code area	446
+1BEh	+446	Partition entry #1	16
+1CEh	+462	Partition entry #2	16
+1DEh	+478	Partition entry #3	16
+1EEh	+494	Partition entry #4	16
+1FEh	+510	55h	2
+1FFh	+511	AAh	
Total size: 446 + 4*16 + 2			512

Structure of a 16-byte Partition Table Entry		
Relative Offsets (within entry)	Length (bytes)	Contents
0	1	Boot Indicator (80h = active)
1 - 3	3	Starting CHS values
4	1	Partition-type Descriptor
5 - 7	3	Ending CHS values
8 - 11	4	Starting Sector
12 - 15	4	Partition Size (in sectors)

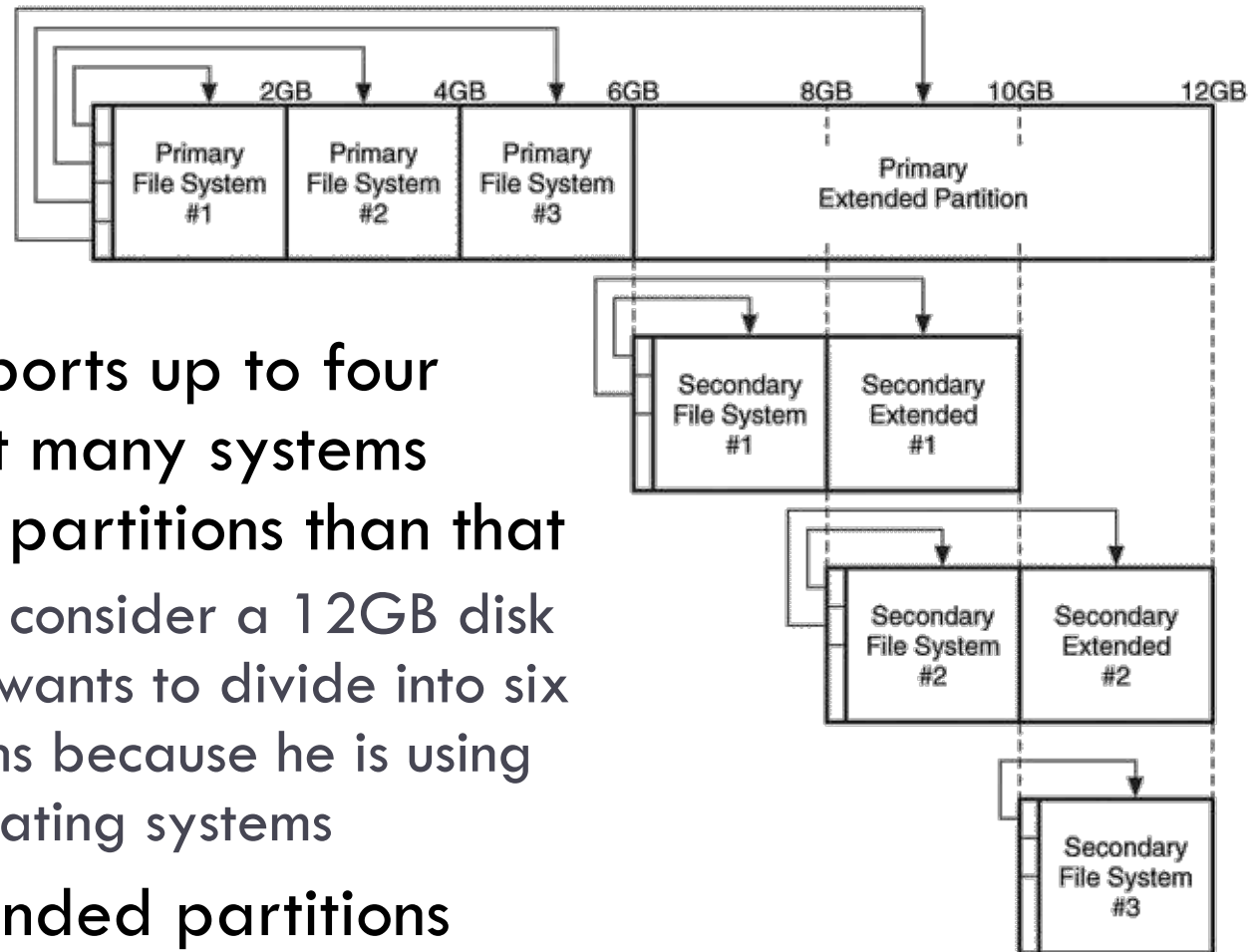


Types of DOS partitions

0	Empty	1e	Hidden W95 FAT1	80	Old Minix	bf	Solaris
1	FAT12	24	NEC DOS	81	Minix / old Lin	c1	DRDOS/sec (FAT-
2	XENIX root	39	Plan 9	82	Linux swap / So	c4	DRDOS/sec (FAT-
3	XENIX usr	3c	PartitionMagic	83	Linux	c6	DRDOS/sec (FAT-
4	FAT16 <32M	40	Venix 80286	84	OS/2 hidden C:	c7	Syrinx
5	Extended	41	PPC PReP Boot	85	Linux extended	da	Non-FS data
6	FAT16	42	SFS	86	NTFS volume set	db	CP/M / CTOS / .
7	HPFS/NTFS	4d	QNX4.x	87	NTFS volume set	de	Dell Utility
8	AIX	4e	QNX4.x 2nd part	88	Linux plaintext	df	BootIt
9	AIX bootable	4f	QNX4.x 3rd part	8e	Linux LVM	e1	DOS access
a	OS/2 Boot Manag	50	OnTrack DM	93	Amoeba	e3	DOS R/O
b	W95 FAT32	51	OnTrack DM6 Aux	94	Amoeba BBT	e4	SpeedStor
c	W95 FAT32 (LBA)	52	CP/M	9f	BSD/OS	eb	BeOS fs
e	W95 FAT16 (LBA)	53	OnTrack DM6 Aux	a0	IBM Thinkpad hi	ee	EFI GPT
f	W95 Ext'd (LBA)	54	OnTrackDM6	a5	FreeBSD	ef	EFI (FAT-12/16/
10	OPUS	55	EZ-Drive	a6	OpenBSD	f0	Linux/PA-RISC b
11	Hidden FAT12	56	Golden Bow	a7	NeXTSTEP	f1	SpeedStor
12	Compaq diagnost	5c	Priam Edisk	a8	Darwin UFS	f4	SpeedStor
14	Hidden FAT16 <3	61	SpeedStor	a9	NetBSD	f2	DOS secondary
16	Hidden FAT16	63	GNU HURD or Sys	ab	Darwin boot	fb	VMware VMFS
17	Hidden HPFS/NTF	64	Novell Netware	b7	BSDI fs	fc	VMware VMKCORE
18	AST SmartSleep	65	Novell Netware	b8	BSDI swap	fd	Linux raid auto
1b	Hidden W95 FAT3	70	DiskSecure Mult	bb	Boot Wizard hid	fe	LANstep
1c	Hidden W95 FAT3	75	PC/IX	be	Solaris boot	ff	BBT



Extended partition concepts



- ▶ The MBR supports up to four partitions, but many systems require more partitions than that
 - ▶ For example, consider a 12GB disk that the user wants to divide into six 2GB partitions because he is using multiple operating systems
- ▶ Solution: extended partitions



Listing the partition contents

► List the partitions in the volume image

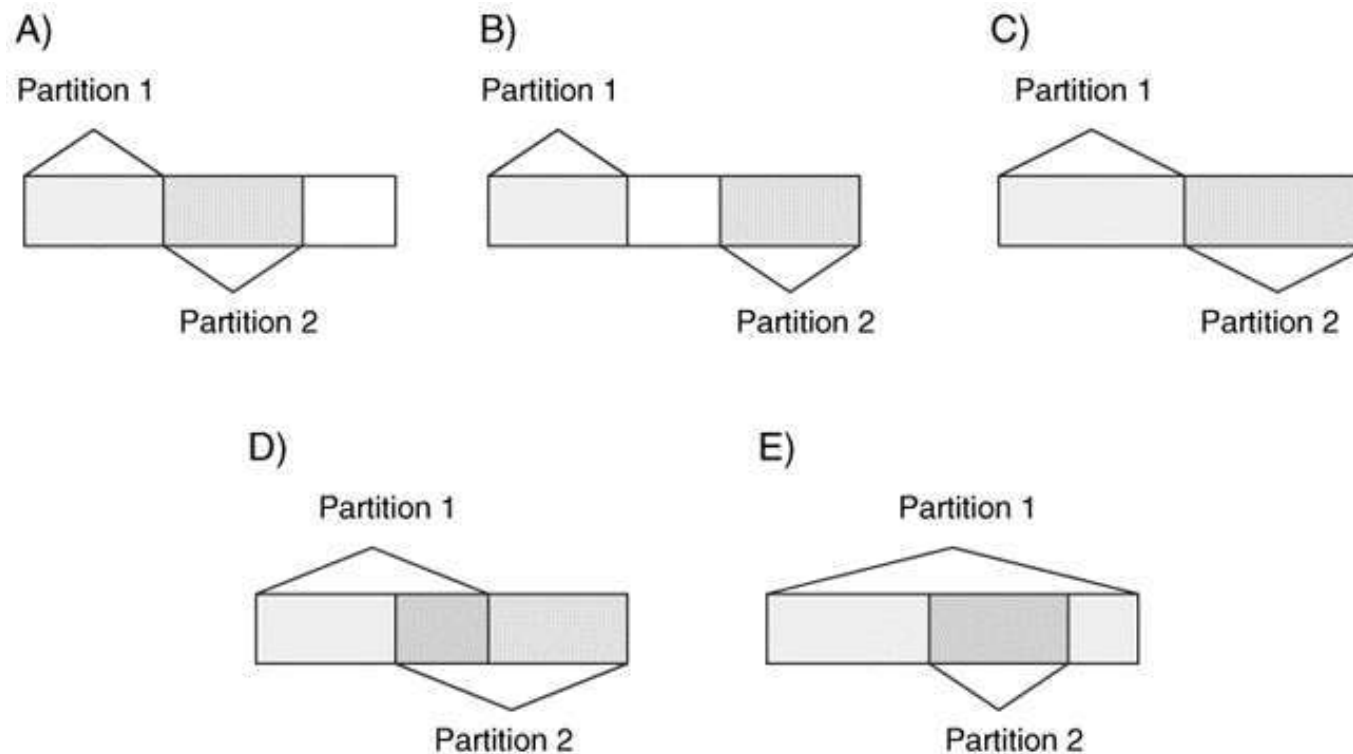
```
# mmls -t dos disk1.dd
Units are in 512-byte sectors
Slot Start End Length Description
00: ----- 0000000000 0000000000 0000000001 Table #0
01: ----- 0000000001 0000000062 0000000062 Unallocated
02: 00:00 0000000063 0001028159 0001028097 Win95 FAT32 (0x0B)
03: ----- 0001028160 0002570399 0001542240 Unallocated
04: 00:03 0002570400 0004209029 0001638630 OpenBSD (0xA6)
05: 00:01 0004209030 0006265349 0002056320 NTFS (0x07)
```

Featured Tool

Forensic Toolkit
(FTK)

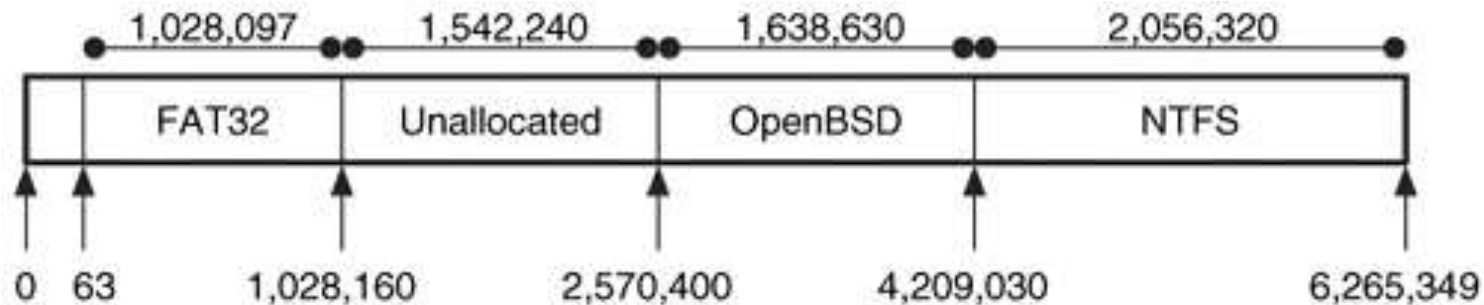
###:###: This format is used with volume systems that have multiple tables. The first two numbers correspond to the table ID and the second set of numbers correspond to the entry in that table. 00:01 is entry 1 in table 0.

- ▶ Check each partition relative to the other partitions
 - ▶ Compares ending of last partition with the end of its parent volume
 - ▶ Compares the start and end sectors of consecutive partitions





Extracting the partition contents



- To extract the file system partitions from the disk image, we take the starting sector and size of each partition as shown here:

```
# dd if=disk1.dd of=part1.dd bs=512 skip=63 count=1028097
# dd if=disk1.dd of=part2.dd bs=512 skip=2570400 count=1638630
# dd if=disk1.dd of=part3.dd bs=512 skip=4209030 count=2056320
```



Recovering deleted partitions

- ▶ Common to thwart a forensic investigation by repartitioning a disk or clearing the partition structures
- ▶ Partition recovery tools work by assuming that a FS was located in each partition
- ▶ Many file systems have data structure that has a signature value
 - ▶ E.g., FAT FS has values 0x55 and 0xAA in bytes 510 and 511 of first sector
- ▶ When the tool finds a signature, additional tests can be conducted on the range of valid values
- ▶ Other tools use heuristics
 - ▶ <http://www.cgsecurity.org/testdisk.html>



Recovering deleted partitions (cont.)

► Example using the gpart tool

```
# gpart -v disk2.dd
* Warning: strange partition table magic 0x0000.
[REMOVED]
Begin scan...
Possible partition(DOS FAT), size(800mb), offset(0mb)
  type: 006(0x06)(Primary 'big' DOS (> 32MB))
  size: 800mb #s(1638566) s(63-1638628)
  chs: (0/1/1)-(101/254/62)d (0/1/1)-(101/254/62)r
  hex: 00 01 01 00 06 FE 3E 65 3F 00 00 00 A6 00 19 00

Possible partition(DOS FAT), size(917mb), offset(800mb)
  type: 006(0x06)(Primary 'big' DOS (> 32MB))
  size: 917mb #s(1879604) s(1638630-3518233)
  chs: (102/0/1)-(218/254/62)d (102/0/1)-(218/254/62)r
  hex: 00 00 01 66 06 FE 3E DA E6 00 19 00 34 AE 1C 00

Possible partition(Linux ext2), size(502mb), offset(1874mb)
  type: 131(0x83)(Linux ext2 filesystem)
  size: 502mb #s(1028160) s(3839535-4867694)
  chs: (239/0/1)-(302/254/63)d (239/0/1)-(302/254/63)r
  hex: 00 00 01 EF 83 FE 7F 2E 2F 96 3A 00 40 B0 0F 00
```



Dealing with full volume encryption

- ▶ **Full volume encryption:** method for encrypting a single partition, either physical or virtual, on a hard drive
- ▶ **Implementations:**
 - ▶ BitLocker
 - ▶ FileVault Disk Encryption
 - ▶ FreeOTFE
 - ▶ TrueCrypt



BitLocker

- ▶ Targets lost-laptop scenario
- ▶ Encrypts NTFS volumes
 - ▶ All disk sectors encrypted with symmetric encryption method
- ▶ Key can be provided by user at boot time
 - ▶ Derived from passphrase
- ▶ Key can be stored in special cryptographic chip that releases it after checking the integrity of the system
 - ▶ Trusted Platform Module (TPM)





BitLocker architecture

- ▶ A BitLocker volume is split into two volumes
 - ▶ Small unencrypted **boot volume**
 - ▶ Large **encrypted volume** for rest of OS & user files
- ▶ Keys
 - ▶ Volume Master Key (VMK)
 - ▶ Unlocked through **authentication procedure**
 - ▶ Full Volume Encryption Key (FVEK)
 - ▶ Used to encrypt sectors of encrypted volume
 - ▶ Stored on boot volume encrypted with VMK
 - ▶ Kept in memory and never written unencrypted to disk





BitLocker startup and operation

- ▶ Authentication procedure checks integrity of system and unseals VMK
- ▶ Integrity check performed by specialized hardware: a TPM chip
- ▶ VMK used to decrypt FVEK, which is kept in main memory
- ▶ For each disk sector accessed
 - ▶ Decrypt on read
 - ▶ Encrypt on write





Hard to recover BitLocker encrypted volumes

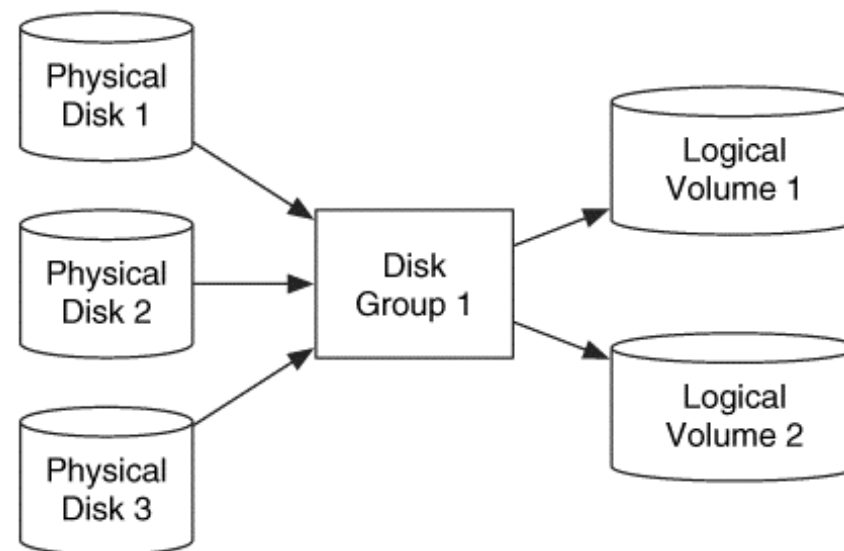
- ▶ **Compromise the TPM**
 - ▶ Based on microprobing the substrate
 - ▶ Requires significant sophistication and specialized instruments
- ▶ **Cold boot attacks**
 - ▶ Volume encryption key is stored in memory to decrypt the drive
 - ▶ RAM retains contents after power down for 2-3 seconds normally
 - ▶ Retention time can be extended for up to an hour by cooling the memory chip
 - ▶ Key recovered by analysing memory



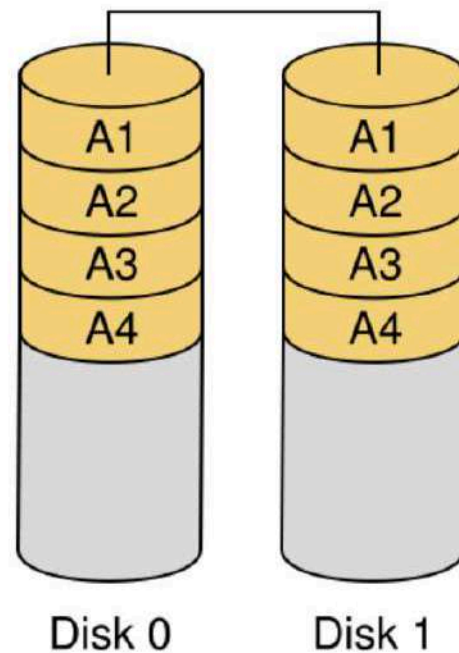


Volume spanning

- ▶ Makes multiple disks appear to be one large disk
 - ▶ Offers no redundancy or performance benefits
- ▶ A **logical volume** is made up of multiple physical disks or partitions that are sequentially merged together



- ▶ RAID = Redundant Arrays of Inexpensive Disks
 - ▶ Multiple RAID levels, each level provides a different amount of reliability and performance improvements

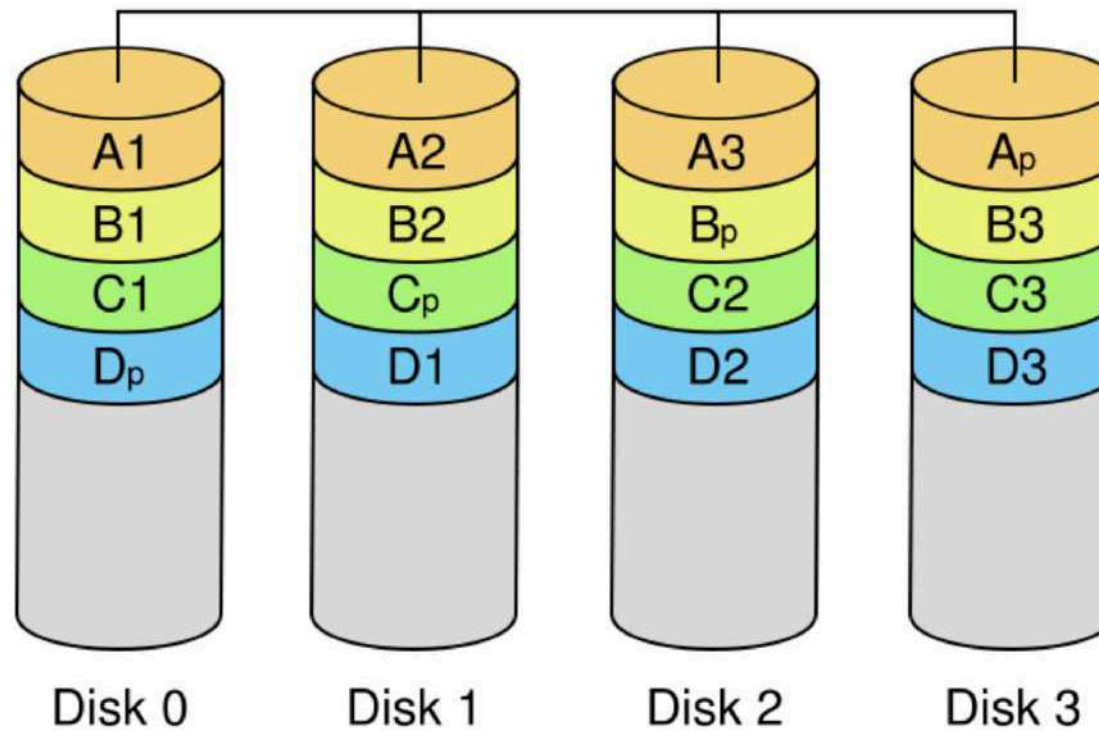


RAID 1 provides mirroring capability



RAID 5

- ▶ In RAID 5 there is no dedicated parity disk, and all disks contain data blocks and parity values on an alternating basis

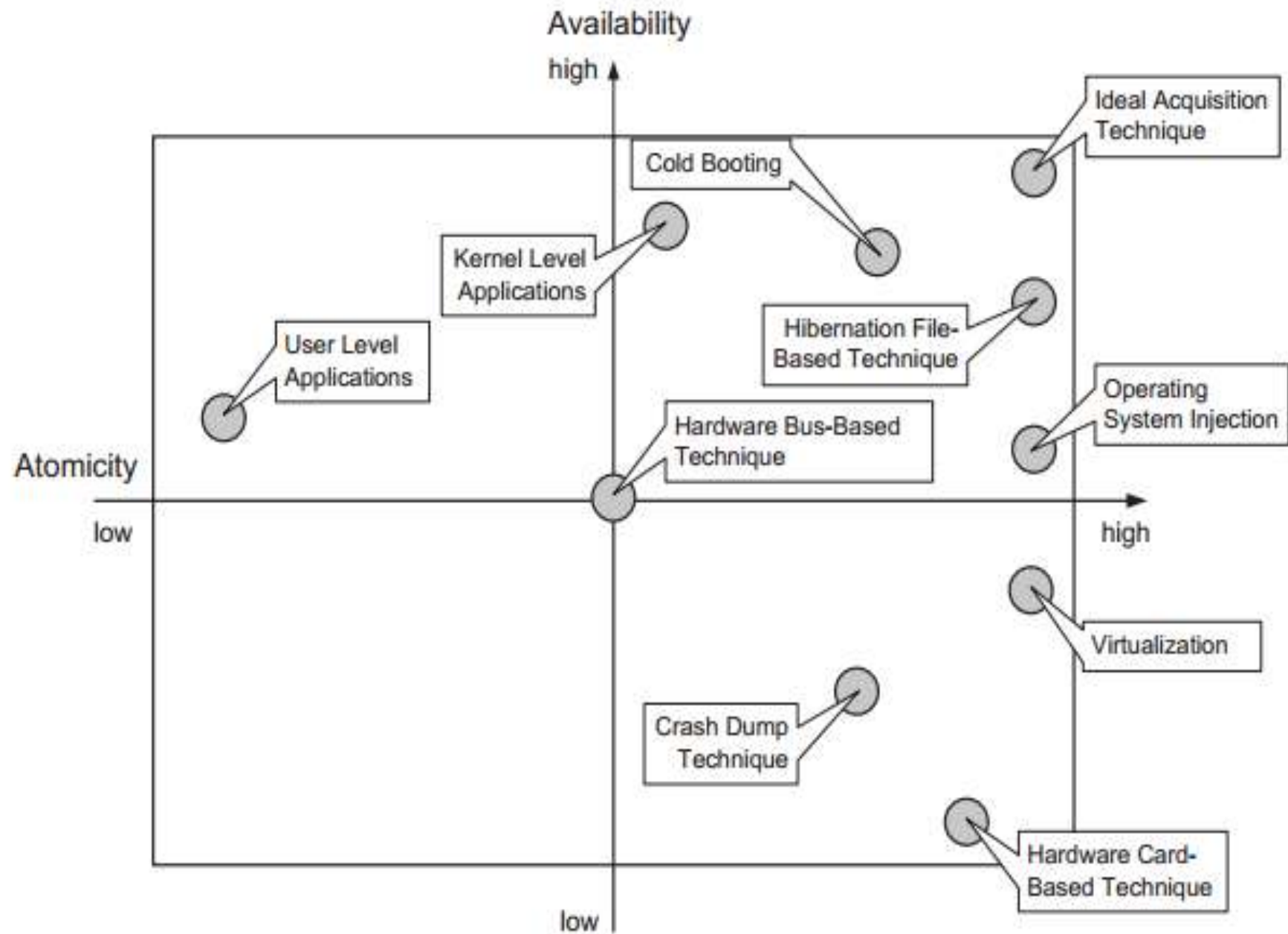


Volatile memory acquisition



Acquisition of volatile memory

- ▶ Techniques for capturing volatile data are divided based on:
 - ▶ Hardware
 - ▶ Software
- ▶ An ideal acquisition method is characterized by both a high atomicity and availability
- ▶ Decision matrix helps investigators in choosing a specific memory acquisition technique





Using a dedicated hardware card

- ▶ Use of special hardware card to obtain forensic image of a computer's RAM
- ▶ It uses Direct Memory Access (DMA)
 - ▶ Hardware Card is installed as a dedicated PCI device and is capable of saving volatile information
- ▶ Limitation: prior installation of PCI card before its use
- ▶ Beneficial when installed on critical servers

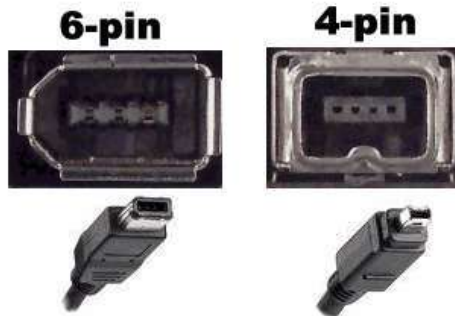




Using a special hardware bus

- ▶ Alternative to PCI cards, several authors suggest reading volatile memory via IEEE 1394 bus - Firewire

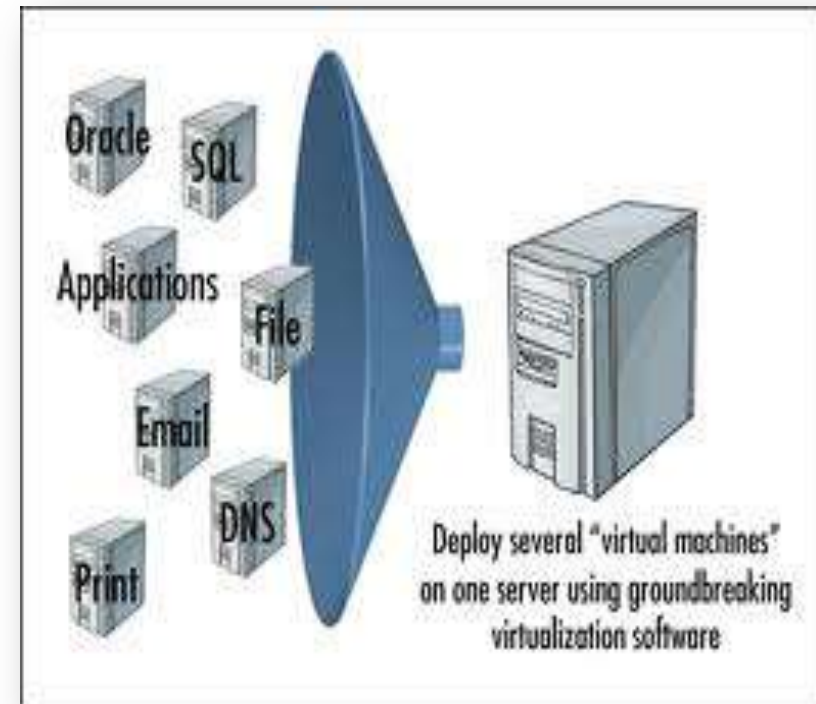
IEEE 1394(AKA Firewire, I-Link)





With the help of virtualization

- ▶ Virtual machine monitor (VMM) is responsible for sharing, managing and restricting access to the hardware resources
- ▶ Exceptional characteristic is capability to be suspended
- ▶ All volatile data is saved in .vmem file
- ▶ With the growing importance of internet-hosted services, investigators have to examine on virtual machines





Using software crash dumps

- ▶ Microsoft windows dump files to hard disk in case of machine failure
- ▶ Preserves the contents of processor registers
- ▶ Dump files can be opened
 - ▶ Debugging Tools
 - ▶ Manually
- ▶ System services may be interrupted
 - ▶ Third party application
 - ▶ Built-in CrashOnCtrlScroll
- ▶ This acquisition technique is more invasive



Other methods

- ▶ **Memory acquisition with user level applications**
 - ▶ E.g., utility to obtain process' environment and state
- ▶ **Memory acquisition with kernel level applications**
 - ▶ E.g., kernel drivers with capability to halt active system processes
- ▶ **Memory acquisition via operating system injection**
 - ▶ Injects OS into the subverted kernel of target machine
- ▶ **Memory acquisition via cold booting**
 - ▶ Restart target machine with a custom kernel to access the retained memory
- ▶ **Memory acquisition using the hibernation file**
 - ▶ Windows Hibernation file: hiberfil.sys



Volatility: a memory analysis framework

- ▶ Volatility modules written in Python
- ▶ Functionality extended by adding plugins
- ▶ Framework implements great parts of concepts and methods outlined
- ▶ Framework
 - ▶ Suitable for high degree forensic tasks
 - ▶ Require high level expertise
 - ▶ Aims at academic researchers and security professionals

Table A.1 – Integral modules of the volatility memory forensics framework

Module name	Description
connections	Locates the <code>_TCBTable</code> hash table in the <code>tcpip.sys</code> driver file and traverses the singly linked list of <code>_TCPT_OBJECT</code> entries to enumerate open network connections on the target system (see Section 4.4).
connscan2	Scans the non-paged pool of the operating system for allocations that contain information about open network connections (see Section 4.4).
dlllist	Retrieves the base address, size, and path of all dynamically loaded libraries (DLLs) that are referenced by a running application. For this operation, the <code>Ldr</code> structure of the Process Environment Block (PEB) is parsed. It stores three doubly linked lists of <code>_LDR_DATA_TABLE_ENTRY</code> types that hold the respective information (see Section 4.5).
files	Retrieves the list of open file handles that is maintained by a process (see Section 4.5).
getsids	Reconstructs the security context of a process to retrieve the list of user and group SIDs (Security Identifiers) the application is associated with (see Section 4.6).
hivelist	Prints the virtual address and name of hive structures that internally represent parts of the system registry (see Section 4.3).
hivescan	Uses a signature-based approach to search for <code>_CMHIVE</code> structures in memory. These data types internally represent parts of the system registry (see Section 4.3).



Conclusions

- ▶ Forensic analysis of any piece of digital evidence is entirely dependent on the ability to interpret how the data is represented
- ▶ Storage media such as hard drives and solid state drives are major sources of data and create multiple challenges for investigators
- ▶ Volume analysis constitutes the first major step at interpreting forensic data from storage media images



References

- ▶ Primary bibliography
 - ▶ Bryan Carrier, *File System Analysis*, 2005:
 - ▶ Chapter 4
 - ▶ Section 1 – Hard Disk Technology



Next class

► II.4 File System Interpretation